

AI CFD Scientist: Toward Open-Ended Computational Fluid Dynamics Discovery with Physics-Aware AI Agents

Nithin Somasekharan Rabi Pathak Manushri Dhanakoti Tingwen Zhang
Ling Yue Andy Zhu Shaowu Pan*
Rensselaer Polytechnic Institute, Troy, NY, USA

Abstract

Recent LLM-based agents have closed substantial portions of the scientific discovery loop in software-only machine-learning research, in chemistry, and in biology. Extending the same loop to high-fidelity physical simulators is harder, because solver completion does not imply physical validity and many failure modes appear only in field-level imagery rather than in solver logs. We present *AI CFD Scientist*, an open-source AI scientist for computational fluid dynamics (CFD) that, to our knowledge, is the first to span literature-grounded ideation, validated execution, vision-based physics verification, source-code modification, and figure-grounded writing within a single inspectable workflow. Three coupled pathways cover parameter sweeps within a fixed solver, case-local C++ library compilation for new physical models, and open-ended hypothesis search against a reference comparator, all running on OpenFOAM through Foam-Agent. At the center of the framework is a vision-language physics-verification gate that inspects rendered flow fields before any result is accepted, rerun, or written into a manuscript. On five tasks under a shared GPT-5.5 backbone, *AI CFD Scientist* autonomously discovers a Spalart–Allmaras runtime correction that reduces lower-wall C_f RMSE against DNS by 7.89% on the periodic hill at $Re_h=5600$; under matched LLM cost, two strong general AI-scientist baselines (ARIS, DeepScientist) execute partial CFD workflows but lack the domain-specific validity gates needed to convert runs into defensible scientific claims; and a controlled planted-failure ablation shows that the vision-language gate detects 14 of 16 silent failures missed by solver-level checks. Code, prompts, and run artifacts are released at <https://github.com/csml-rpi/cfd-scientist>.

1 Introduction

Large language model agents have closed substantial portions of the scientific discovery loop in software-only machine-learning research [19, 38], in chemistry [3], and in biology [23]. Extending these systems to physical sciences whose evidence comes from high-fidelity simulators is the next frontier and remains underexplored, in part because the discovery loop interacts with the simulator at a level deeper than text-mediated tool use.

Computational fluid dynamics (CFD) makes this loop particularly strict for three reasons. First, solver completion does not imply physical validity: a case can run cleanly while still using the wrong geometry, missing a key flow feature, or producing degenerate output. These failure modes are

*Corresponding author: pans2@rpi.edu

typically invisible to solver logs.² Second, validity gates are themselves scientific objects: mesh independence and reference-data alignment must be confirmed before any claim, not assumed. Third, the closure model is a research variable, edited at the C++ level rather than swapped in a config, so source-code modification is part of the hypothesis space rather than a configuration option.

Two lines of work approach this loop from opposite sides but neither covers it end-to-end. Generic AI-scientist frameworks [25, 32, 38, 40] automate ideation, code, plotting, and writing, but they were designed for software-only ML workflows and lack the physical-validity gates that distinguish a runnable simulation from a defensible scientific claim. CFD-specific agents [7, 12, 33, 41] automate case setup, execution, and parts of post-processing on OpenFOAM-style substrates, but stop short of the full discovery loop. The closest related system, *turbulence.ai* [12], frames an AI scientist for fluid mechanics that formulates ideas, orchestrates experiments, and drafts reports, yet remains closed-source and, based on public documentation as of submission, does not expose a vision-language physics-verification gate, a mesh-independence gate, or open-ended source-level discovery as first-class subsystems.

We present *AI CFD Scientist*, an open-source AI scientist for CFD that, to our knowledge, is the first to span literature-grounded ideation, validated execution, vision-based physics verification, source-code modification, and figure-grounded writing within a single inspectable workflow. The framework runs on OpenFOAM through Foam-Agent [41] and exposes three coupled pathways: regular experimentation through parameter sweeps within a fixed solver, source-code modification that compiles case-local C++ libraries for new physical models, and open-ended hypothesis search that autonomously edits source code and coefficients against a reference comparator. At the center of the framework is a vision-language physics-verification gate that inspects rendered flow fields before any result is accepted, rerun, or written into a manuscript: a subsystem absent from the AI-scientist baselines we compare against. The architecture follows five operational design principles distilled from CFD practice, detailed in section 3.

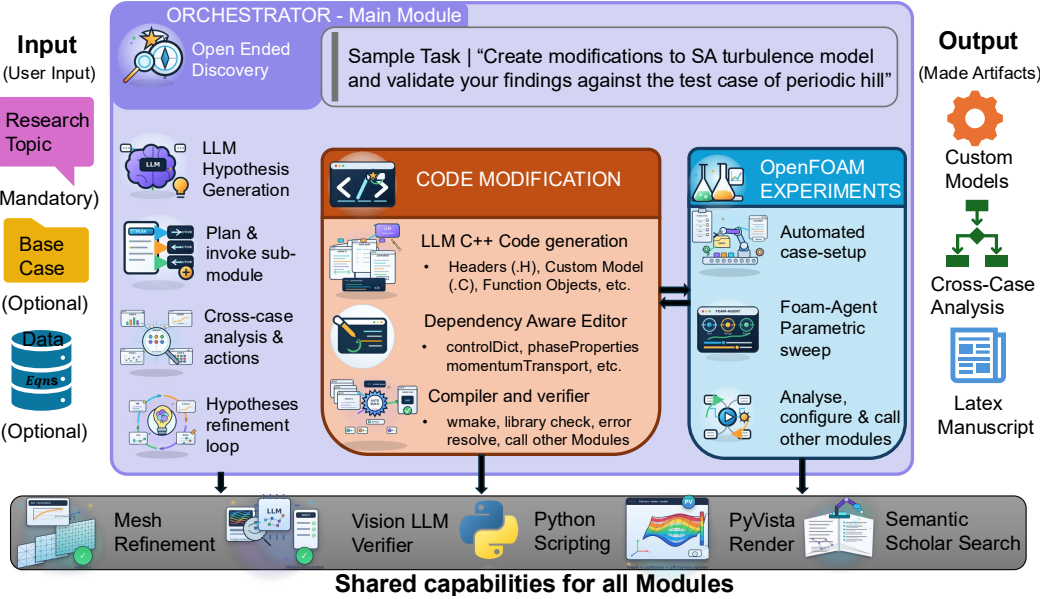


Figure 1: Architecture of *AI CFD Scientist*. A natural-language topic, optional base case, and optional reference data is passed as input to the framework. Three first-class pathways execute under a shared capability bus: (i) *regular experimentation* via literature-aware ideation, requirement validation, mesh-independence gating, and Foam-Agent execution; (ii) *code modification* that patches and compiles case-local C++ model libraries; (iii) *open-ended discovery* that wraps both modules in an outer hypothesis loop. A VLM physics gate inspects rendered flow fields before any result is accepted, rerun, or written.

²For example, a backward-facing-step case can converge cleanly while a reattachment-length extractor returns a wrong-sign value: invisible in the solver log, but obvious in a C_f plot.

On five tasks under a shared GPT-5.5 backbone, *AI CFD Scientist* executes regular experimentation, custom-model compilation, and open-ended discovery; in the open-ended task, the system autonomously discovers a Spalart–Allmaras runtime correction that reduces lower-wall C_f RMSE against DNS by 7.89% on the periodic hill at $Re_h=5600$. Under matched LLM cost, two strong general AI-scientist baselines (ARIS [40], DeepScientist [32]) execute partial CFD workflows but lack the domain-specific validity gates needed to convert runs into defensible scientific claims. A controlled planted-failure ablation shows that the vision-language physics gate detects 14 of 16 silent failures missed by solver-level checks.

Table 1: Positioning *AI CFD Scientist* against generic AI-scientist frameworks and CFD-specific agents.

System	Literature Survey	Novelty Filtering	CFD Execution	Mesh Independence Study	Simulator Source Code Editing	VLM-Based Physics Check	Paper Generation	Reference Data Ingestion
<i>Generic AI-scientist frameworks designed primarily for ML research</i>								
AI Scientist (v2)[38]	✓	✗	✗	✗	✗	✗	✓	✗
Agent Laboratory[25] / AgentRxiv[24]	✓	✗	✗	✗	✗	✗	✓	✗
AI co-scientist[16]	✓	✗	✗	✗	✗	✗	✗	✗
CycleResearcher[31]	✓	✗	✗	✗	✗	✗	✓	✗
DeepScientist[32]	✗	✗	✓	✗	✗	✗	✗	✗
ARIS[40]	✗	✗	✓	✗	✓	✗	✗	✗
<i>CFD-specific agents</i>								
MetaOpenFOAM[5], ChatCFD[10], OpenFOAMGPT[22]	✗	✗	✓	✗	✗	✗	✗	✗
Foam-Agent [41]	✗	✗	✓	✗	✗	✗	✗	✗
CFDagent[37] / SwarmFoam[39] / PhyNiKCE[11] / CFD-copilot[8]	✗	✗	✓	✗	✗	✗	✗	✗
<i>turbulence.ai</i> [12]	✓	✗	✓	✗	✗	✗	✓	✗
FlamePilot[33]	✓	✗	✗	✗	✗	✗	✗	✗
<i>AI CFD Scientist (this work)</i>	✓	✓	✓	✓	✓	✓	✓	✓

2 Related Work

Robot scientists and autonomous laboratories. Closing the scientific loop predates LLMs. The *Robot Scientist* systems [18, 28] demonstrated end-to-end hypothesis generation and physical experimentation in molecular biology, and symbolic-regression engines such as Eureka [26] automated equation discovery from data. More recent self-driving laboratories [15, 20, 27] fuse robotic experimentation with Bayesian-optimization planners. These systems target chemistry, materials, and biology, where ground truth comes from physical measurement; they do not transfer to CFD, where validity depends on closure choices, mesh resolution, and physical interpretation of computed fields rather than wet-lab readouts.

LLM-based AI-scientist frameworks. A second wave of systems closes the same loop in pure software using LLMs. *The AI Scientist* and *AI Scientist-v2* [19, 38] produce end-to-end ML papers from a research idea; *Agent Laboratory* and *AgentRxiv* [24, 25] formalize multi-agent collaboration and inter-paper memory; *AI co-scientist* [16] layers critique-driven refinement. *CycleResearcher*, *AI-Researcher*, and *Zochi* [17, 30, 31] emphasize iterative refinement and tool-use; *DeepScientist* [32] and *ARIS* [40] are the most recent strong baselines, both built around long-context execution loops, and are the two systems used in our head-to-head comparison. Domain instances exist in chemistry and biology, example: *ChemCrow*, autonomous chemistry agents, and CRISPR-GPT [1, 3, 23]. Evaluation infrastructure (Bohrium–SciMaster, AstaBench, PaperBench, MLR-Bench [2, 4, 29, 42]) scores artifact quality on ML research workflows.

CFD- and OpenFOAM-specific agents. A parallel line of work targets CFD itself. PythonFOAM and foamlib [14, 21] expanded the Python surface for case manipulation and in-situ analysis. LLM-centered systems then moved from prompt assistance to structured orchestration: FoamPilot [36]

and AutoCFD [9] are early prompt-driven assistants, OpenFOAMGPT and MetaOpenFOAM (with optimized variants) [5–7, 13, 22] structure the case-authoring workflow, and Foam-Agent [41] adds RAG-based retrieval and a reviewer loop. ChatCFD [10], CFDagent [37], SwarmFoam [39], PhyNiKCE [11], CFD-copilot [8], *turbulence.ai* [12], and FlamePilot [33] extend the surface to chat-driven workflows, multi-agent decomposition, physics constraints, and combustion. General coding agents also solve a subset of OpenFOAM workflows by reusing tutorials [34], and a separate line asks whether LLMs can act as neural fluid surrogates [35]. None of these systems combine all the relevant features needed for automating CFD discovery. This gap motivates *AI CFD Scientist*.

3 CFD Scientist

AI CFD Scientist encodes CFD discovery as a set of expert-written prompts, guidelines, and execution pathways rather than a generic chat loop. We provide two implementations: a checkpointed LangGraph workflow for end-to-end orchestration, and a modular skills-based version whose components can be reused inside other orchestrators. In both forms, agents exchange structured artifacts such as study JSON, requirement paragraphs, source-edit plans, run directories, figure manifests, interpretation JSON, and manuscript drafts as shown in figure 1. The design follows five principles distilled from CFD practice: **(P1)** physical validity is not log-readable, so image-level inspection is mandatory; **(P2)** source code modification is a research object rather than a configuration option; **(P3)** mesh independence is a required convergence gate; **(P4)** agents must not hallucinate an alternate experiment, swap the swept variable, or relax success criteria in order to make a failing case easier to run; **(P5)** every claim in the generated manuscript must trace back to a specific figure, numerical value, or interpretation record produced by a case that passed its validity gates, never to the model’s prior knowledge.

Three pathways. *Regular experimentation:* This pathway runs CFD simulation studies without modifying simulator source code. Given a research topic, the literature-aware ideation agent retrieves Semantic Scholar records, synthesizes candidate gaps, and emits a structured study JSON. A string-similarity novelty filter rejects near-duplicate ideas and triggers re-prompting when needed. The specification agent then converts each experiment into a single-paragraph requirement. A validator checks solver availability, time-control consistency, boundary-condition completeness, and unit consistency; failed specifications are rewritten through a repair prompt. Validated requirements are passed to Foam-Agent [41], which generates the case dictionaries, executes the simulation, and performs low-level error correction. *Code modification:* for studies that require a model not present in the OpenFOAM source code, an expert-written code-mod agent generates C++ source and dictionary edits, compiles a *case-local* library under `{case}/customModels/`, and uses compiler diagnostics as structured feedback; a smoke test verifies the library loads and produces interpretable fields before any sweep. *Open-ended discovery:* given an abstract goal such as *find a novel turbulence-model modification that better matches a given DNS reference*, or any user-supplied objective with a comparator, an outer hypothesis loop autonomously generates and tests candidate ideas without further human input. At each iteration it proposes a concrete edit (a source-code change to the turbulence model, a coefficient or parameter adjustment, or a new diagnostic script), invokes the code-modification and regular-experimentation pathways to compile and run it as a real OpenFOAM case, and compares the resulting flow field against both the reference data and the unmodified baseline. Iterations are scored by a user-specified comparator, checkpointed and promoted only when the score improves over baseline.

Mesh-independence gate. A baseline mesh is selected from a starter case, literature, or generated by Foam-Agent. A refined mesh is constructed with $\sim 10\%$ near-wall and $\sim 5\%$ bulk refinement, preserving topology, blocking, and meshing method. Baseline and refined cases run with identical models/BCs/numerics; local fields and surface/global metrics (U, p, C_f, C_p , lift/drag/ Δp) are compared, percent differences tabulated, and a 5% threshold flags QoIs that require Richardson/GCI escalation.

VLM physics-verification gate (the central evidence gate, implementing P1). After a case finishes running, an *interpreter agent* reads the case directory and the requirement, and emits a diagnostic plan, deciding the physical quantities to visualize and compare against reference data if provided. Then a visualization creator agent writes a PyVista and/or matplotlib script that extracts the

relevant diagnostic fields, and renders them as PNGs. The rendered visualizations are then handed to a VLM in two separate calls. The first call is a *quality filter*: it checks whether figures are readable; failures are redrawn. The second call is the *physics check*: the VLM inspects the accepted figures, looking for the expected flow features, and judges if the image is consistent with the experiment requirement. It further drives the rerun controller and the writer. The gate exists because a case can pass every log-based check, completed time-stepping, no warnings, while still using the wrong geometry, missing important flow features or instantiating a degenerate custom model. These are exactly the failure modes a log-only interpreter cannot catch, and *none of the AI-scientist frameworks in Table 1 expose this gate as a first-class subsystem*. Section G gives the failure-mode taxonomy that motivates these gates.

Rerun controller and writer loop (P4, P5). When a gate rejects a run, the rerun controller revises the requirement. It may reuse nearby successful cases, such as relaxation factors, or schemes. After all cases pass their gates, an analysis agent generates paper-ready cross-case figures, distinct from the diagnostic visualizations used during verification. The writer then receives the literature bundle, study JSON, per-case requirements, source-edit history, figure manifest, and analysis text. It drafts LaTeX, compiles the manuscript, receives critique from a reviewer agent on formatting, claim–evidence alignment, reference coverage, and redundancy, and revises until acceptance or budget exhaustion.

4 Experiments: *AI CFD Scientist* with GPT-5.5

Setup. *AI CFD Scientist* is run end-to-end with GPT-5.5. All evaluation is manual because no automated CFD-paper rubric currently scores the workflows the system produces.

Tasks. We execute five CFD tasks summarized in Table 2: **T1**) BFS turbulence-model sensitivity at $Re_h=25,400$, **T2**) jet/plume oscillation across Reynolds numbers, **T3**) custom non-Newtonian viscosity in a channel, **T4**) a custom Spalart–Allmaras (SA) modifier for the periodic hill, and **T5**) open-ended discovery of an SA modification that improves lower-wall C_f agreement with DNS. The first two use the regular-experimentation pathway, the next two use the simulator source-code modification pathway, and the final task uses the open-ended discovery pathway. Detailed experiment matrices and per-case quantitative tables are reported in Section B; token usage and estimated cost are reported in Section I.

Table 2: *AI CFD Scientist* GPT-5.5 task overview. Pathway: REG = regular experimentation; CM = code modification; OED = open-ended discovery.

ID	Task	Path	Cases run	Custom code compiled	Headline <i>AI CFD Scientist</i> result (GPT-5.5)
T1	BFS turbulence sens. ($Re_h=25.4k$)	REG	4 RANS	none	Runs 4 RANS closures on the same backward-facing-step mesh.
T2	Jet/plume Re-sweep ($Re=60-600$)	REG	7 transient	none	Recovers the expected centreline U_x scaling across most cases.
T3	Custom viscosity (channel)	CM	6	<code>libcustomViscosity</code>	Autonomously writes and compiles a power-law viscosity library, validates it against the Newtonian limit ($n=1$).
T4	Custom SA modifier (periodic hill, $Re_h=10,595$)	CM	6	<code>libCustomSA</code>	Compiles a custom Spalart–Allmaras modifier and compares against baseline and reference data.
T5	Open-ended SA discovery (periodic hill, $Re_h=5600$)	OED	44 iterations	coded <code>fvModels</code>	Autonomously discovers a quadrupolar SA runtime correction that reduces lower-wall C_f RMSE versus DNS by 7.89% ($0.004297 \rightarrow 0.003958$)

4.1 Findings across the five GPT-5.5 case studies

T1 — BFS turbulence sensitivity. *AI CFD Scientist* planned a four-model matrix (standard $k-\epsilon$, realizable $k-\epsilon$, $k-\omega$ SST, SA) at $Re_h=25,400$, ran each through the mesh independence study (26.9k–38.1k cells), and rendered diagnostic contours. The VLM check flagged a sign-convention / origin error in the reattachment extractor and triaged a $k-\epsilon$ output as inconsistent with separated-flow physics; the SST and SA closures produced the most plausible recirculation topology in streamlines (Figure 2a). The intended behavior was confirmed: rather than rank closures from a post-processor

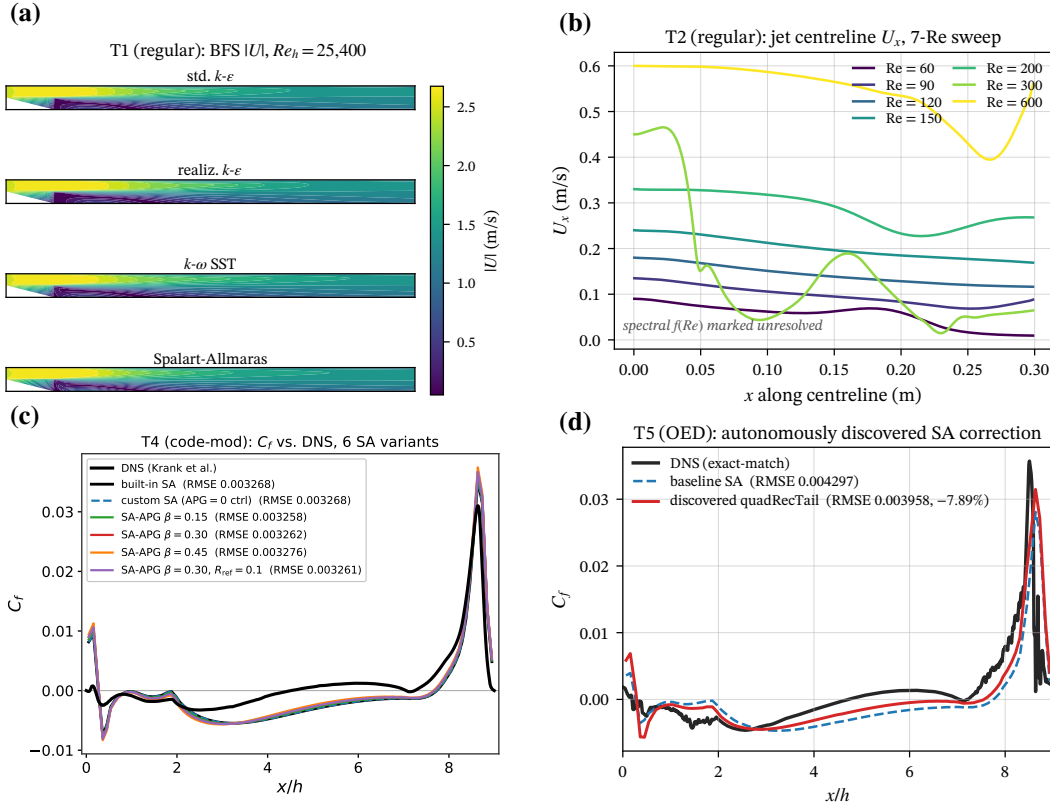


Figure 2: Representative quantities of interest from the case studies. (a) T1: BFS $|U|$ contours across four RANS closures at $Re_h=25,400$; recirculation-zone differences are visible behind the step. (b) T2: centreline U_x profiles across the 7-Re jet sweep, showing the recovered velocity scaling and emerging instability at higher Re . (c) T4: lower-wall C_f overlay against reference for four APG-modifier SA variants and control (APG=0) at $Re_h=10,595$. (d) T5: autonomously discovered quadrupolar SA correction at iter_044 reduces C_f RMSE against DNS by 7.89% at $Re_h=5600$ on the periodic hill.

known to be buggy, the system flagged the QoI and abstained. The input topic given to *AI CFD Scientist* is provided in [section A](#). No baseline OpenFOAM files or reference data are provided.

T2 — Jet/plume Re sweep. Seven 2D laminar jet cases on identical 35,156-cell meshes ran end-to-end. Centreline velocity scaling was recovered ($U_{c,max}$ tracks bulk velocity from 0.09 to 0.60 m/s as Re sweeps $60 \rightarrow 600$, with oscillations emerging at high Re , [Figure 2b](#)), and case-006 was flagged as anomalous (centreline-mean collapse). The input topic given to *AI CFD Scientist* is provided in [section A](#). No baseline OpenFOAM files or reference data are provided.

T3 — Custom viscosity (code modification). The code-modification agent generated a generalized-Newtonian viscosity model $\nu(\dot{\gamma}) = \nu_\infty + k \max(\dot{\gamma}, \dot{\gamma}_{min})^{n-1}$ as case-local source files and compiled the custom viscosity library on the first attempt. Six cases executed to steady state. With $n=1$ the custom law reproduced the parabolic Newtonian baseline (centreline within 0.5% of the analytic 1.5 m/s); centreline velocity varied $\sim 3.8\%$ across the sweep (1.4542–1.5231 m/s). The input topic given to *AI CFD Scientist* is provided in [section A](#). Baseline OpenFOAM files for Newtonian channel flow are provided.

T4 — Custom SA modifier (code modification). A SA variant with an adverse-pressure-gradient (APG) correction multiplier on the production term was compiled into `libCustomSA.so`. Six cases (1 APG=0 control + 4 APG variants) ran on an identical mesh. The control case matched the built-in SA baseline to four decimals ($U_{max}=1.5959$ m/s in both), validating that the custom code path does not perturb the underlying solver; the APG sweep then induced a $\sim 1.25\%$ U_{max} sensitivity (1.5759–1.5959 m/s), and C_f overlays against reference data were rendered for all six variants ([Figure 2c](#)). The input topic given to *AI CFD Scientist* is provided in [section A](#). Baseline Periodic Hill flow OpenFOAM files are provided to the framework along with reference DNS data.

T5 — Open-ended SA discovery. Given the periodic hill at $Re_h=5600$, a starter SA case, reference wall friction coefficient (C_f) data, and the objective “minimize lower-wall C_f RMSE,” *AI CFD Scientist* ran 44 discovery iterations (worked-example trace in Figure 3). The discovered model adds an implicit source to the SA $\tilde{\nu}$ equation,

$$S_{\text{extra}} = [C_{\text{rec}} G_{\text{rec}} - C_{\text{sink}} G_{\text{sink}} + C_{\text{src}} G_{\text{src}} - C_{\text{tail}} G_{\text{tail}}] |\nabla \mathbf{U}| \tilde{\nu},$$

with each $G_*(x, y_w) = \exp[-\frac{1}{2}(x - x_*)^2/\sigma_*^2] \exp(-y_w/L_{y,*})$ a wall-normalized Gaussian patch. The best iteration ($C_{\text{rec}}=2.12$, $C_{\text{sink}}=2.25$, $C_{\text{src}}=1.2$, $C_{\text{tail}}=0.75$) reduces C_f RMSE against DNS from 0.004297 (baseline SA) to 0.003958, a 7.89% **reduction** (Figure 2d). The model is delivered as a coded `fVModels` block. The full 44-iteration discovery trajectory, the discovered `quadRecTail` coefficient table, and an OpenFOAM source excerpt are in Section C. The input topic given to *AI CFD Scientist* is provided in section A.

Multi-agent collaboration in CFD Scientist - open-ended discovery example

Discovering a novel quadrupolar Spalart-Allmaras runtime correction (quadRecTail) for the periodic-hill benchmark at $Re_h = 5600$

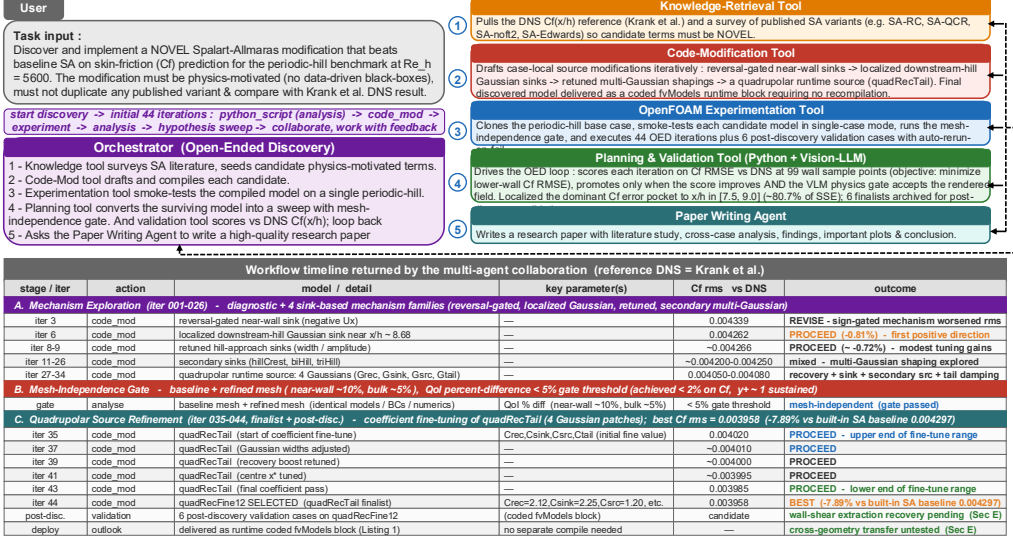


Figure 3: **Worked example of the open-ended-discovery (OED) pathway on T5** (periodic hill, $Re_h=5600$). *Top*: the five-step multi-agent collaboration under the OED orchestrator — knowledge retrieval (1), code modification (2), single-case smoke test (3), mesh-independence-gated execution (4), and paper writing (5) — with one orchestrator-issued tool call shown per box. *Bottom*: the 44-iteration trajectory grouped by mechanism family. **Block A** (iter 001–026) traverses four sink-based families (reversal-gated, localized Gaussian, retuned hill-approach, secondary multi-Gaussian) before introducing a quadrupolar runtime source at iter 027–034. **Block B** confirms mesh independence on the baseline + refined (~10% near-wall, ~5% bulk) chain (achieved < 2% on C_f , $y^+ \sim 1$). **Block C** fine-tunes the quadrupolar coefficients (iter 035–043) and selects `iter_044_quadRecFine12` (`quadRecTail`), which reduces lower-wall C_f RMSE against DNS from 0.004297 (baseline SA) to 0.003958, a –7.89% improvement. The discovered model is delivered as a coded `fVModels` runtime block requiring no recompilation; cross-geometry transfer remains untested. Full trajectory, discovered coefficients, and OpenFOAM source in Section C.

Further details on each case can be found in section E and the shortcoming discussed in section F.

4.2 VLM physics-verification gate: planted-failure ablation

The VLM physics-verification gate is intended to catch failures that are not reliably visible from solver completion alone. We evaluate this role with a controlled planted-failure ablation.

Setup. We start from four production-passed template cases, one each from the jet, BFS, periodic-hill, and channel studies. For each case, we apply one file-system-level perturbation from a four-category failure taxonomy: `missing_deliverable`, `wrong_magnitude_metric`, `broken_postprocessing`, and `convergence_not_settled`. This gives $4 \times 4 = 16$ planted

failures, plus four clean controls. The verifier is the same single-shot vision-LLM call used in production. A case is counted as flagged if the verifier returns either REVISE or RERUN. Using planted failures rather than rerunning the full system gives deterministic ground-truth labels and isolates the sensitivity of the VLM gate from solver noise. The design matrix and per-case archive are provided in [Section J](#).

Table 3: Planted-failure ablation for the VLM physics-verification gate. A case is counted as detected when the verifier returns REVISE or RERUN.

Failure category	Detected	Interpretation
missing_deliverable	4/4	Requested output is absent, although the case may still complete.
wrong_magnitude_metric	4/4	Existing output contradicts the requested or physically plausible magnitude.
broken_postprocessing	4/4	Output files contain zero, NaN, or otherwise degenerate values.
convergence_not_settled	2/4	Shortened runs can appear visually complete when endTime is edited consistently with the truncated state.
All planted failures	14/16	The gate catches most non-log-readable failures.

As shown in [Table 3](#), the gate detects 14/16 planted failures. It catches all missing-deliverable, wrong-magnitude, and broken-postprocessing cases, which are failures that can pass solver-level checks but invalidate interpretation. The main weakness is convergence sufficiency: only 2/4 truncated-run cases are flagged because edited endTime values of the cases can make incomplete simulations appear visually complete.

5 Cross-Framework Comparison: *AI CFD Scientist* vs. ARIS vs. DeepScientist

The five-task study above evaluates *AI CFD Scientist* in isolation. To separate the effect of CFD-specific gates from generic AI-scientist scaffolding, we compare against ARIS [30] and DeepScientist [32] on T1–T4 under the same GPT-5.5 backbone. T5 is excluded because neither baseline supports open-ended source-level discovery. Evaluation is manual and artifact-based, using archived case directories, solver logs, custom C++ libraries, figures, and reports. [Table 4](#) reports capability coverage; [Table 5](#) reports per-task quality. Cost, token usage, and a per-task evidence ledger are provided in [Sections D](#) and [I](#).

Table 4: Capability comparison on T1–T4 with a shared GPT-5.5 backbone, supported by inspection of archived artifacts.

Capability (under GPT-5.5)	ARIS	DeepScientist	<i>AI CFD Scientist</i>
Literature retrieval (Semantic Scholar / OpenAlex / arXiv)	✗	○	✓
Novelty filter against retrieved literature	✗	✗	✓
Requirement validation and repair before execution	○	○	✓
OpenFOAM execution end-to-end	✓	✓	✓
Mesh-independence gate	✗	✗	✓
Case-local custom-model compilation (T3 and T4)	✓	✓	✓
VLM-based physics-verification gate	✗	✗	✓
DNS / reference-data alignment for C_f	✓	✗	✓
Cross-case analysis with paper-ready figures	○	○	✓
Figure-grounded LaTeX writer with reviewer loop	✗	○	✓
Conservative <i>unresolved</i> verdict when evidence is incomplete	○	✗	✓

Reading the rubric. Two patterns stand out. First, ARIS and DeepScientist often execute simulations and produce clean trends, but they lack the CFD-specific gates needed to decide whether those trends are scientifically supported. On T1 and T2, for example, they report closure rankings or $St(Re)$ correlations despite missing mesh or reference-data evidence. *AI CFD Scientist* is more conservative: when evidence is incomplete, it records an *unresolved* verdict rather than converting a runnable case into a scientific claim.

Table 5: Per-task quality rubric on T1–T4 under matched GPT-5.5. **S**=strong, **P**=partial, **W**=weak, **X**=absent or stalled. Row blocks: *TIQ* = task-implementation quality; *SRQ* = scientific-research quality.

Axis	Framework	T1 (BFS turb.)	T2 (Jet Re-sweep)	T3 (Custom ν)	T4 (Custom SA)
TIQ	ARIS	P (3 closures executed, no mesh-indep.)	P (5-Re sweep, fixed mesh)	P (1 custom variant compiled)	P (custom SA compiled, DNS C_f acknowledged, no manuscript)
	DeepScientist	P (3 closures, controlled comparison)	P (5-Re sweep, $f \propto Re$)	P (1 custom variant + technical report)	P (custom SA compiled and executed; partial report)
	<i>AI CFD Scientist</i>	S (4 closures, mesh-gate, VLM-triaged)	S (7-Re sweep on uniform mesh, conservative)	S (5-variant sweep + Newtonian degeneracy)	S (validated code path; DNS overlayed and used; LaTeX draft)
SRQ	ARIS	W (closure ranking issued without DNS / experimental validation)	W ($St \approx 0.019$ fit reported without grid-convergence or DNS check)	W (no DNS or experimental comparison)	W (no APG=0 control; no result analysis)
	DeepScientist	W (closure ranking issued without DNS validation)	W ($St \approx 0.031$ fit reported without validation)	P (technical report; no DNS or experimental comparison)	W (no APG=0 control; no result analysis)
	<i>AI CFD Scientist</i>	P (VLM-flagged post-processor; closure ranking explicitly withheld)	P (analysis agent marks $f(Re)$ unresolved on missing metadata)	P (Newtonian degeneracy validated; remaining gaps preserved in writer)	P (APG=0 control validated; quantitative ranking reported, differences marginal)
OEI	ARIS	X (no idea generation)	X (sweep follows prompt only)	X (single variant)	W (one physics-motivated SA mod)
	DeepScientist	X	X	X	W (one β variant beyond default)
	<i>AI CFD Scientist</i>	P (lit-grounded multi-axis sweep)	P (lit-grounded sweep + perturbation BCs)	P (5-variant ($k, n, \nabla p$) sweep)	P (5-variant sweep + control)

Second, the distinction does not lie in whether each framework can compile a case-local custom model — all three did, on both T3 and T4 — but in how completely the surrounding scientific pipeline is exercised. ARIS and DeepScientist each ran one custom variant against a reference and reported a markdown summary; *AI CFD Scientist* additionally ran an APG=0 control case to validate the custom code path, produced a DNS overlay against the reference, and emitted a figure-grounded LaTeX draft. The comparison therefore suggests that the advantage is not in source-level editing per se, but in the surrounding CFD-specific scientific control flow.

6 Conclusion

AI CFD Scientist is, to our knowledge, the first open-source AI scientist for CFD that closes the discovery loop from a natural-language topic to a manuscript draft. Unlike generic AI-scientist frameworks or CFD agents focused mainly on case generation and execution, *AI CFD Scientist* integrates literature-grounded ideation, novelty filtering, mesh-independence gating, source-level model modification, VLM-based physics verification, reference-data alignment, and figure-grounded writing. Across five CFD tasks, it supports regular experimentation, source-code modification, and open-ended discovery; in one discovery study, it identifies a Spalart–Allmaras runtime correction that reduces lower-wall C_f RMSE against DNS by 7.89% on the periodic hill at $Re_h=5600$. Under matched conditions, other generic scientist frameworks execute parts of the same workflows but do not provide the combined CFD-specific control flow needed for physically grounded automation. We release *AI CFD Scientist* with code, prompts, and run artifacts as a community baseline for CFD-specific scientific automation.

Limitations and scope. The results are encouraging but bounded in scope. (i) *Single backbone*: all numbers use GPT-5.5 (Codex); LLM sweeps and additional baselines are deferred for cost. (ii) *Manual evaluation for cross-framework comparison*: no automated CFD-paper rubric scores these workflows, so Table 5 reflects expert artifact reading. The framework is supervised scientific assistance, not unattended publication.

References

- [1] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023. doi: 10.1038/s41586-023-06792-0. URL <https://www.nature.com/articles/s41586-023-06792-0>.
- [2] Jonathan Bragg et al. AstaBench: Rigorous benchmarking of AI agents with a scientific research suite. *arXiv preprint arXiv:2510.21652*, 2025. doi: 10.48550/arXiv.2510.21652. URL <https://arxiv.org/abs/2510.21652>.
- [3] Andres M. Bran, Sam Cox, Olivier Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Chemcrow: Augmenting large-language models with chemistry tools. *Nature Machine Intelligence*, 2024. doi: 10.48550/arXiv.2304.05376. URL <https://arxiv.org/abs/2304.05376v5>.
- [4] Hui Chen, Miao Xiong, Yujie Lu, Wei Han, Ailin Deng, Yufei He, Jiaying Wu, Yibo Li, Yue Liu, and Bryan Hooi. Mlr-bench: Evaluating ai agents on open-ended machine learning research. *arXiv preprint arXiv:2505.19955*, 2025. doi: 10.48550/arXiv.2505.19955. URL <https://arxiv.org/abs/2505.19955>.
- [5] Yuxuan Chen, Xu Zhu, Hua Zhou, and Zhuyin Ren. Metaopenfoam: An llm-based multi-agent framework for cfd. *arXiv preprint arXiv:2407.21320*, 2024. doi: 10.48550/arXiv.2407.21320. URL <https://arxiv.org/abs/2407.21320>.
- [6] Yuxuan Chen, Long Zhang, Xu Zhu, Hua Zhou, and Zhuyin Ren. Optmetaopenfoam: Large language model driven chain of thought for sensitivity analysis and parameter optimization based on cfd. *arXiv preprint arXiv:2503.01273*, 2025. doi: 10.48550/arXiv.2503.01273. URL <https://arxiv.org/abs/2503.01273>.
- [7] Yuxuan Chen, Xu Zhu, Hua Zhou, and Zhuyin Ren. Metaopenfoam 2.0: Large language model driven chain of thought for automating cfd simulation and post-processing. *arXiv preprint arXiv:2502.00498*, 2025. doi: 10.48550/arXiv.2502.00498. URL <https://arxiv.org/abs/2502.00498>.
- [8] Zhehao Dong, Shanghai Du, Zhen Lu, and Yue Yang. Cfd-copilot: Leveraging domain-adapted large language model and model context protocol to enhance simulation automation. *arXiv preprint arXiv:2512.07917*, 2025. doi: 10.48550/arXiv.2512.07917. URL <https://arxiv.org/abs/2512.07917>.
- [9] Zhehao Dong, Zhen Lu, and Yue Yang. Fine-tuning a large language model for automating computational fluid dynamics simulations. *Theoretical and Applied Mechanics Letters*, 15:100594, 2025. doi: 10.1016/j.taml.2025.100594. URL <https://doi.org/10.1016/j.taml.2025.100594>.
- [10] E. Fan, Weizong Wang, and Tianhan Zhang. Chatecf: An end-to-end cfd agent with domain-specific structured thinking. *Advanced Intelligent Discovery*, 2025. doi: 10.1002/aidi.202500174. URL <https://arxiv.org/abs/2506.02019>.
- [11] E. Fan, Lisong Shi, Zhengtong Li, and Chih-Yung Wen. Phynikce: A neurosymbolic agentic framework for autonomous computational fluid dynamics. *arXiv preprint arXiv:2602.11666*, 2026. doi: 10.48550/arXiv.2602.11666. URL <https://arxiv.org/abs/2602.11666>.
- [12] Jingsen Feng, Yupeng Qi, Ran Xu, Sandeep Pandey, and Xu Chu. turbulence.ai: an end-to-end ai scientist for fluid mechanics. *Theoretical and Applied Mechanics Letters*, page 100620, 2025. ISSN 2095-0349. doi: 10.1016/j.taml.2025.100620. URL <https://www.sciencedirect.com/science/article/pii/S2095034925000522>.
- [13] Jingsen Feng, Ran Xu, and Xu Chu. Openfoamgpt 2.0: End-to-end, trustworthy automation for computational fluid dynamics. *International Journal of Heat and Fluid Flow*, 2026. doi: 10.1016/j.ijheatfluidflow.2026.110399. URL <https://arxiv.org/abs/2504.19338>.
- [14] Gabriel S. Gerlero and Pablo A. Kler. foamlib: A modern Python package for working with OpenFOAM. *Journal of Open Source Software*, 10(109):7633, 2025. doi: 10.21105/joss.07633. URL <https://doi.org/10.21105/joss.07633>.
- [15] Aldair E. Gongora, B. Xu, W. Perry, C. Okoye, P. Riley, K. G. Reyes, E. F. Morgan, and K. A. Brown. A bayesian experimental autonomous researcher for mechanical design. *Science Advances*, 6(15): eaaz1708, 2020. doi: 10.1126/sciadv.aaz1708. URL <https://www.science.org/doi/10.1126/sciadv.aaz1708>.
- [16] Juraj Gottweis et al. Towards an ai co-scientist. *arXiv preprint arXiv:2502.18864*, 2025. doi: 10.48550/arXiv.2502.18864. URL <https://arxiv.org/abs/2502.18864>.

- [17] Intology. Zochi technical report. GitHub repository and technical report, 2025. URL <https://github.com/IntologyAI/Zochi>.
- [18] Ross D. King et al. The automation of science. *Science*, 324(5923):85–89, 2009. doi: 10.1126/science.1165620. URL <https://www.science.org/doi/10.1126/science.1165620>.
- [19] Cong Lu, Chris Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL <https://arxiv.org/abs/2408.06292>.
- [20] Benjamin P. MacLeod et al. A self-driving laboratory advances the Pareto front for material properties. *Nature Communications*, 13:995, 2022. doi: 10.1038/s41467-022-28580-6. URL <https://www.nature.com/articles/s41467-022-28580-6>.
- [21] Romit Maulik, Dimitrios K. Fytanidis, Bethany Lusch, Venkatram Vishwanath, and Saumil Patel. Python-FOAM: In-situ data analyses with OpenFOAM and Python. *Journal of Computational Science*, 62:101750, 2022. doi: 10.1016/j.jocs.2022.101750. URL <https://doi.org/10.1016/j.jocs.2022.101750>.
- [22] Sandeep Pandey, Ran Xu, Wenkang Wang, and Xu Chu. Openfoamgpt: A rag-augmented llm agent for openfoam-based computational fluid dynamics. *Physics of Fluids*, 2025. doi: 10.1063/5.0257555. URL <https://arxiv.org/abs/2501.06327>.
- [23] Yuanhao Qu, Kaixuan Huang, Ming Yin, Kanghong Zhan, Dyllan Liu, Di Yin, Henry C. Cousins, William A. Johnson, Xiaotong Wang, Mihir Shah, Russ B. Altman, Denny Zhou, Mengdi Wang, and Le Cong. Crispr-gpt for agentic automation of gene-editing experiments. *Nature Biomedical Engineering*, 10(2):245–258, February 2026. ISSN 2157-846X. doi: 10.1038/s41551-025-01463-z. URL <https://doi.org/10.1038/s41551-025-01463-z>.
- [24] Samuel Schmidgall and Michael Moor. Agentrxiv: Towards collaborative autonomous research. *arXiv preprint arXiv:2503.18102*, 2025. doi: 10.48550/arXiv.2503.18102. URL <https://arxiv.org/abs/2503.18102>.
- [25] Samuel Schmidgall et al. Agent laboratory: Using LLM agents as research assistants. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025. doi: 10.18653/v1/2025.findings-emnlp.320. URL <https://arxiv.org/abs/2501.04227>.
- [26] Michael Schmidt and Hod Lipson. Distilling free-form natural laws from experimental data. *Science*, 324(5923):81–85, 2009. doi: 10.1126/science.1165893. URL <https://www.science.org/doi/10.1126/science.1165893>.
- [27] Martin Seifrid et al. Autonomous chemical experiments: challenges and perspectives on establishing a self-driving lab. *Accounts of Chemical Research*, 55(17):2454–2466, 2022. doi: 10.1021/acs.accounts.2c00220. URL <https://doi.org/10.1021/acs.accounts.2c00220>.
- [28] Andrew Sparkes, Wayne Aubrey, Emma Byrne, Amanda Clare, Muhammed Nadeem Khan, Maria Liakata, Magdalena Markham, Jem J. Rowland, Larisa N. Soldatova, Ken E. Whelan, Michael Young, and Ross D. King. Towards robot scientists for autonomous scientific discovery. *Automated Experimentation*, 2:1, 2010. doi: 10.1186/1759-4499-2-1. URL <https://doi.org/10.1186/1759-4499-2-1>.
- [29] Giulio Starace et al. Paperbench: Evaluating AI’s ability to replicate machine learning research. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. doi: 10.48550/arXiv.2504.01848. URL <https://arxiv.org/abs/2504.01848>.
- [30] Jiabin Tang, Lianghao Xia, Zhonghang Li, and Chao Huang. AI-Researcher: Autonomous scientific innovation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. doi: 10.48550/arXiv.2505.18705. URL <https://openreview.net/forum?id=kQWY0YUAC4>.
- [31] Yixuan Weng, Minjun Zhu, Guangsheng Bao, Hongbo Zhang, Jindong Wang, Yue Zhang, and Linyi Yang. Cyclereviewer: Improving automated research via automated review. In *International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2411.00816. URL <https://openreview.net/forum?id=bjcsVLoHYs>.
- [32] Yixuan Weng, Minjun Zhu, Qiuji Xie, Qiyao Sun, Zhen Lin, Sifan Liu, and Yue Zhang. Deepscientist: Advancing frontier-pushing scientific findings progressively. In *International Conference on Learning Representations (ICLR)*, 2026. doi: 10.48550/arXiv.2509.26603. URL <https://openreview.net/forum?id=cZFgsLq8Gs>.

- [33] Ke Xiao, Haoze Zhang, Runze Mao, Han Li, and Zhi X. Chen. Towards llm-enabled autonomous combustion research: A literature-aware agent for self-corrective modeling workflows. *arXiv preprint arXiv:2601.01357*, 2026. doi: 10.48550/arXiv.2601.01357. URL <https://arxiv.org/abs/2601.01357>.
- [34] Ke Xiao, Haoze Zhang, Yangchen Xu, Runze Mao, Han Li, and Zhi X. Chen. A preliminary assessment of coding agents for CFD workflows. *arXiv preprint arXiv:2602.11689*, 2026. doi: 10.48550/arXiv.2602.11689. URL <https://arxiv.org/abs/2602.11689>.
- [35] Qisong Xiao, Xinhai Chen, Qinglin Wang, Xiaowei Guo, Binglin Wang, Weifeng Chen, Zhichao Wang, Yunfei Liu, Rui Xia, Hang Zou, Gencheng Liu, Shuai Li, and Jie Liu. LLM4Fluid: Large language models as generalizable neural solvers for fluid dynamics. *arXiv preprint arXiv:2601.21681*, 2026. doi: 10.48550/arXiv.2601.21681. URL <https://arxiv.org/abs/2601.21681>.
- [36] Leidong Xu, Danyal Mohaddes, and Yi Wang. LLM agent for fire dynamics simulations. *arXiv preprint arXiv:2412.17146*, 2024. doi: 10.48550/arXiv.2412.17146. URL <https://arxiv.org/abs/2412.17146>.
- [37] Zhaoyue Xu, Long Wang, Chunyu Wang, Yixin Chen, Qingyong Luo, Hua-Dong Yao, Shizhao Wang, and Guowei He. Cfdagent: A language-guided, zero-shot multi-agent system for complex flow simulation. *Physics of Fluids*, 2025. doi: 10.1063/5.0294696. URL <https://arxiv.org/abs/2507.23693>.
- [38] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025. doi: 10.48550/arXiv.2504.08066. URL <https://arxiv.org/abs/2504.08066>.
- [39] Chunwei Yang, Yankai Wang, Jianxiang Tang, Haojie Qu, Ziqiang Zou, Yu Liu, Chunrui Deng, Zhifang Qiu, and Ming Ding. Swarmfoam: An openfoam multi-agent system based on multiple types of large language models. *arXiv preprint arXiv:2601.07252*, 2026. doi: 10.48550/arXiv.2601.07252. URL <https://arxiv.org/abs/2601.07252>.
- [40] Ruofeng Yang, Yongcan Li, and Shuai Li. Aris: Fully autonomous research via adversarial multi-agent collaboration, 2026. URL <https://github.com/wanshuiyin/Auto-claude-code-research-in-sleep>.
- [41] Ling Yue, Nithin Somasekharan, Tingwen Zhang, Yadi Cao, Shimin Di, and Shaowu Pan. Foam-agent 2.0: An end-to-end composable multi-agent framework for automating cfd simulation in openfoam. *arXiv preprint arXiv:2509.18178*, 2025. doi: 10.48550/arXiv.2509.18178. URL <https://arxiv.org/abs/2509.18178>.
- [42] Linfeng Zhang et al. Bohrium + SciMaster: Building the infrastructure and ecosystem for agentic science at scale. *arXiv preprint arXiv:2512.20469*, 2025. doi: 10.48550/arXiv.2512.20469. URL <https://arxiv.org/abs/2512.20469>.

A Input topic for T1-T5

T1: BFS turbulence-model sensitivity (REG)

Study the sensitivity of turbulence-model choice on the backward-facing step at $Re_h = 25\{, \}400$ (Driver and Seegmiller geometry and conditions). Run four RANS closures: $k\text{--}\epsilon$, $k\text{--}\omega$ SST, Spalart--Allmaras, and a Reynolds-stress model. All four are run on the same mesh-independence-passed mesh, with identical inlet profiles, BCs, time controls, and post-processing. For every run, generate streamlines through the recirculation region, C_f along the bottom wall, and station-wise mean-velocity and Reynolds-stress profiles downstream of the step. Compare each model's reattachment length and recirculation topology against the experimental dataset and report which closure best reproduces the measured flow.

T2: Jet/plume Reynolds-number sweep (REG)

Perform a Reynolds-number sweep on a 2D plane jet/plume across $Re \in [60, 600]$ using 7 cases (logarithmically spaced). The goal is to characterize (a) the scaling of the centreline streamwise velocity U_x and (b) the dominant oscillation-frequency dependence $f(Re)$. Use a transient solver, integrate long enough to obtain statistically converged statistics, and produce centreline U_x profiles, contour snapshots at representative Re , and a per-case estimate of the dominant oscillation frequency together with an honest assessment of whether that frequency is resolved at the chosen mesh.

T3: Custom power-law viscosity on a channel (CM)

Implement a custom non-Newtonian power-law viscosity model $\mu_{eff} = k |\dot{\gamma}|^{n-1}$ as a `\emph{case-local}` OpenFOAM viscosity library (do not edit the OpenFOAM source tree). Validate it on a fully-developed pressure-driven 2D channel flow. First run a Newtonian reference case ($n = 1$) and verify that the custom library reproduces the built-in Newtonian solution to within numerical tolerance. Then sweep over $(k, n, \nabla p)$ at 5 representative operating points and report, for each case, the velocity profile, wall shear stress, and effective-viscosity field, together with the expected shear-thinning or shear-thickening trend.

T4: Custom SA modifier on the periodic hill (CM)

Implement a custom modification to the Spalart--Allmaras (SA) turbulence model that changes `\emph{only}` the production term and leaves every other SA term unchanged. The original SA production term is
%
\[
C_{b1} \backslash, \tilde{S} \backslash, \tilde{\nu},
\]


```

%
which is replaced by
%
\[
p_{\mathrm{mult}} \cdot C_{b1} \sqrt{S} \sqrt{\nu},
\quad
p_{\mathrm{mult}} \sqrt{\Omega} - R_{\mathrm{ref}} \sqrt{\Omega} \left( 1 + \beta \sqrt{\frac{S}{\Omega}} \right),
\quad
p_{\min}, \quad p_{\max}
\]
%
where
\((S = \sqrt{2} \sqrt{|\nabla \mathbf{u}|})\) is the
strain-rate magnitude,
\((\Omega = \sqrt{2} \sqrt{|\nabla \mathbf{u}|})\) is the
rotation-rate magnitude,
and the constants take the fixed values
\((\beta = 6.0)\), \((R_{\mathrm{ref}} = 0.82)\), \((p_{\min} = 0.05)\), \((p_{\max} = 5.0)\).
Compile this modification as a case-local OpenFOAM library (do not edit the
OpenFOAM source tree) and validate it on the periodic hill at
 $Re_h = 10^5$ . First run a zero-perturbation control case with  $\beta = 0$ 
(which gives  $p_{\mathrm{mult}} \equiv 1$ ) and confirm that the modified
solver matches the built-in SA model exactly on this mesh. Then run a 5-
point
sensitivity sweep over  $\beta$  around the nominal value  $\beta = 6.0$ , and
report for each case the change in  $U_{\max}$ , shifts in separation and
reattachment locations, and the lower-wall  $C_f$  difference relative to the
control.

```

T5: Open-ended SA discovery against DNS (OED)

Open-ended discovery objective: starting from the standard Spalart--Allmaras model on the periodic hill at $Re_h = 5600$, autonomously explore modifications to the SA closure that reduce the lower-wall skin-friction (C_f) RMSE against the provided DNS reference. The system may (i) modify the source-level SA equations and recompile a case-local library, (ii) add runtime correction terms via `fvModels` without recompilation, (iii) adjust SA model coefficients, or (iv) introduce new geometry-aware or flow-aware sensitization. Score every iteration by C_f RMSE versus DNS, promote an iteration only if its score improves over the unmodified-SA baseline **and** the VLM physics gate accepts the rendered flow field, and finally report the best-performing modification together with its mechanism.

B Per-Task Experiment Matrices and Quantitative Results

This appendix gives the GPT-5.5 experiment configurations and per-case quantitative metrics behind [Table 2](#) and the findings in [Section 4.1](#).

B.1 T1 — Backward-facing step turbulence-model sensitivity

Table 6: T1 experiment matrix and per-case metrics. $Re_h=25,400$, step height $h=0.01$ m.

Case	Model	Mesh (cells)	Final t	$U_{\max}$ (m/s)	x_r/h (extracted)	VLM-gate verdict
case_001	standard $k-\varepsilon$	30,548	2000	1.6013	sign-anomaly −0.0332	flagged: post-processor sign-error in C_f extractor; closure ranking withheld
case_002	realizable $k-\varepsilon$	29,400	2000	1.6297	sign-anomaly −0.0383	flagged: identical extracted value as cases 003/004 indicates artifact
case_003	$k-\omega$ SST	26,960	994	1.6256	sign-anomaly −0.0383	accepted topology (most plausible recirculation); ranking withheld until QoI repaired
case_004	Spalart–Allmaras	38,068	2000	1.6084	sign-anomaly −0.0383	accepted topology; ranking withheld

B.2 T2 — Jet/plume oscillation Reynolds-number sweep

Table 7: T2 experiment matrix and per-case metrics. Identical 35,156-cell mesh across all cases; slot width $w=0.01$ m; $\nu=1.5 \times 10^{-5}$ m²/s; antisymmetric inlet perturbation 1% for first 0.05 s. Spectral metrics are marked *unresolved* due to cross-experiment metadata-parser failure (Section F).

Case	Re	$U_{\text{mag,max}}$	$U_{c,\text{max}}$	$\bar{U}_c/U_{c,\text{max}}$	Status	Notes
001	60	0.0902	0.0900	0.599	unresolved	monotonic baseline; spectral metadata not recovered
002	90	0.1352	0.1350	0.719	unresolved	monotonic baseline
003	120	0.1801	0.1800	0.791	unresolved	monotonic baseline
004	150	0.2402	0.2400	0.838	unresolved	monotonic baseline
005	200	0.3301	0.3300	0.868	unresolved	monotonic baseline
006	300	0.5117	0.4654	0.301	flagged anomaly	centreline-mean collapse; deflection / unsteady state suspected
007	600	0.6004	0.6000	0.904	unresolved	only ~ 2 s available, weakening spectral confidence

B.3 T3 — Custom viscosity model on a channel

Table 8: T3 experiment matrix and per-case metrics. Generalized-Newtonian viscosity $\nu(\dot{\gamma}) = \nu_\infty + k \max(\dot{\gamma}, \dot{\gamma}_{\min})^{n-1}$. Periodic channel, length 2.0 m, half-height 0.05 m, $\nu_{\text{ref}}=0.01$ m²/s. Custom library compiled case-local (no edits to OpenFOAM tree).

Case	Variant	k	n	$U_{c,\text{max}}$ (m/s)	Role / verdict
001	Newtonian reference ($n=1$)	—	1.0	1.4925	baseline; matches analytic 1.5 within 0.5%
002	custom (best)	1×10^{-3}	0.6	1.4698	shear-thinning, intermediate
003	custom k_{low}	5×10^{-4}	0.6	1.4800	shear-thinning, lower k
004	custom k_{high}	2×10^{-3}	0.6	1.4542	lowest $U_{c,\text{max}}$ (effective viscosity up)
005	custom n_{low}	1×10^{-3}	0.3	1.4741	stronger shear-thinning
006	custom n_{high}	1×10^{-3}	1.2	1.5231	shear-thickening; highest $U_{c,\text{max}}$

B.4 T4 — Custom Spalart–Allmaras modifier on the periodic hill

Table 9: T4 experiment matrix. Periodic hill, $Re_h=10,595$, identical mesh across cases. Custom library `libCustomSA.so` compiled case-local. DNS reference: Krank et al. (2018), 1153 wall points at matched Re . C_f RMSE computed over matched x/h domain.

Case	Variant	β	R_{ref}	C_f RMSE	x_r/h	Role / verdict
—	DNS (Krank et al.)	—	—	—	4.51	reference
001	built-in SA baseline	—	—	0.003268	7.73	reference
002	custom SA (APG=0 control)	0	0	0.003268	7.73	matches baseline; <i>validates custom code path</i>
003	SA-APG $\beta=0.15$	0.15	0.05	0.003258	7.70	best RMSE (marginal)
004	SA-APG $\beta=0.30$	0.30	0.05	0.003262	7.68	APG variant
005	SA-APG $\beta=0.45$	0.45	0.05	0.003276	7.66	shortest recirculation
006	SA-APG $\beta=0.30$, $R_{\text{ref}}=0.10$	0.30	0.10	0.003261	7.68	R_{ref} sensitivity

B.5 T5 — Open-ended SA discovery (overview)

The full 44-iteration trajectory is in [Section C](#); the headline finalist is `iter_044_quadRecFine12` with C_f RMSE vs. DNS 0.003958 versus baseline SA 0.004297 (a 7.89% reduction, $Re_h=5600$).

C Open-Ended Discovery: Trajectory and Discovered Model

C.1 Discovery objective and reference

The discovery objective was to minimize the RMSE of the lower-wall skin-friction coefficient C_f along 99 wall sample points against an exact-match DNS reference. The dominant baseline-SA error is concentrated in the outlet hill-approach region ($x/h \in [7.5, 9.0]$, $\sim 80.7\%$ of total SSE), with a positive C_f overshoot near $x/h \approx 8.64$ – 8.72 . Baseline separation/reattachment estimates ($x/h=0.269$ / 7.753) deviate from DNS (0.191 / 4.726); the discovery target is C_f RMSE only, not separation/reattachment location.

C.2 Iteration trajectory

Table 10: T5 OED trajectory milestones. Score is C_f RMSE vs. DNS exact-match reference; lower is better. Status: **REVISE** = score worsened; **PROCEED** = score improved and gates accepted. Baseline SA: 0.004297.

Iteration block	Mechanism family proposed	pro-posed	Best score in block	Status	Rationale / observation
iter_001–005	diagnostic only (no source)	—	—	—	Localized dominant C_f error to outlet ($x/h \in [7.5, 9.0]$, 80.7% SSE)
iter_003	reversal-gated sink (negative U_x)	near-wall	0.004339	REVISE	Sign-gated mechanism worsened RMSE
iter_006	localized downstream-hill Gaussian sink near $x/h \approx 8.68$	downstream-hill	0.004262	PROCEED (−0.81%)	First positive direction
iter_008–009	retuned hill-approach sinks (width / amplitude)	sinks	~ 0.004266	PROCEED ($\sim -0.72\%$)	Modest tuning gains
iter_011–026	secondary sinks (hillCrest, biHill, triHill)	—	~ 0.004200 – 0.004250	mixed	Multi-Gaussian shaping explored
iter_027–034	quadrupolar source (4 Gaussians)	runtime	0.004050– 0.004080	PROCEED	Recovery boost + sink + secondary source + tail damping
iter_035–043	introduced quadrupolar fine-tuning	coefficient	0.003985– 0.004020	PROCEED	Convergence on coefficient region
iter_044	quadRecFine12 (lected)	(se-)	0.003958	PROCEED (−7.89% vs. baseline)	Best iteration; promoted to artifact

C.3 Discovered quadRecTail model: form and coefficients

The discovered model adds an implicit source to the SA $\tilde{\nu}$ equation,

$$S_{\text{extra}}(x, y_w) = [C_{\text{rec}} G_{\text{rec}}(x, y_w) - C_{\text{sink}} G_{\text{sink}}(x, y_w) + C_{\text{src}} G_{\text{src}}(x, y_w) - C_{\text{tail}} G_{\text{tail}}(x, y_w)] |\nabla \mathbf{U}| \tilde{\nu},$$

with each Gaussian patch $G_*(x, y_w) = \exp[-\frac{1}{2}(x - x_*)^2/\sigma_*^2] \exp(-y_w/L_{y,*})$, and the coefficients in Table 11. The four terms have distinct physical interpretations: a broad recovery-region production boost (G_{rec}), a localized sink that suppresses the dominant outlet C_f overshoot (G_{sink}), a narrow secondary production trigger upstream of the sink (G_{src}), and a tail-region damping patch that controls residual overshoot near the outlet (G_{tail}).

Table 11: Discovered quadRecTail coefficients (iter_044_quadRecFine12). Values are read directly from the archived `oed_artifact.json`.

Patch	Amplitude	x_* (x/h)	σ_*	$L_{y,*}$	Physical role
G_{rec} (recovery boost)	$C_{\text{rec}}=2.12$	6.00	2.36	0.228	adds production in the broad recovery region $x/h \approx 3-7$ where SA underpredicts wall shear
G_{sink} (sink)	$C_{\text{sink}}=2.25$	8.69	0.085	0.045	suppresses excessive $\tilde{\nu}$ in the dominant C_f -overshoot region $x/h \approx 8.5-8.8$
G_{src} (secondary src.)	$C_{\text{src}}=1.20$	8.43	0.05	0.04	narrow upstream production trigger that prevents the sink from over-correcting
G_{tail} (tail damping)	$C_{\text{tail}}=0.75$	8.86	0.12	0.07	damps residual positive C_f overshoot near the outlet $x/h \approx 8.7-9.0$

C.4 Deployment as a coded fvModels block

The discovered model is delivered as a coded fvModels runtime block, requiring no separate compilation. The implicit source $K(x, y_w) = (\sum_i C_i G_i) |\nabla U|$ is added through `fvm::Sp(K, eqn.psi())`, which keeps the modification implicit in the SA $\tilde{\nu}$ equation (Listing 1).

Listing 1: Excerpt of the coded fvModels runtime source delivered as the discovered model artifact (constant/fvModels block).

```
customSource
{
    type            coded;
    selectionMode    all;
    field            nuTilda;
    C_rec    2.12;  xRec    6.0;  sigmaRec    2.36;  LyRec    0.228;
    C_src    1.2;   xSrc    8.43; sigmaSrc    0.05;  LySrc    0.04;
    C_sink    2.25; xSink    8.69; sigmaSink    0.085; LySink    0.045;
    C_tail    0.75; xTail    8.86; sigmaTail    0.12; LyTail    0.07;
    codeAddSup
    #{
        // assemble K = [C_rec*G_rec + C_src*G_src - C_sink*G_sink - C_tail*G_tail]
        * |grad U|
        // per cell from yWall and cell centres (omitted: G_* Gaussian patches, |
        grad U|),
        // then add implicitly to the SA \tilde{\nu} equation
        const volScalarField K = /* ...assembled per-cell as above... */;
        eqn += fvm::Sp(K, eqn.psi());
    };
}
```

D Cross-Framework Evidence Ledger

This appendix backs the rubric in Table 5 with the artifact evidence each framework produced under matched GPT-5.5 on the four standard tasks. Numbers are read directly from each framework’s run archive.

Table 12: T1 (BFS turbulence sensitivity) artifact evidence under GPT-5.5.

Framework	Cases run	Mesh (cells)	Reattachment x_r/h extracted	Validation / paper artifact
ARIS	3 RANS ($k-\varepsilon$, SST, SA)	7,040	6.99, 7.84, 7.76	summary.md + CSV; no DNS / experimental overlay; no manuscript
DeepScientist	3 RANS ($k-\varepsilon$, SST, SA)	8,800	6.55, 7.35, 6.95	summary.md / paper outline only; no DNS / experimental overlay
AI CFD Scientist	4 RANS (+ re-alizable $k-\varepsilon$)	26.9k–38.1k	flagged sign-anomaly; ranking withheld	VLM-flagged C_f post-processor; LaTeX paper draft, mesh-gate report

Table 13: T2 (jet/plume Re-sweep) artifact evidence under GPT-5.5.

Framework	Cases run	Mesh (cells)	Reported correlation	Validation / paper artifact
ARIS	5 ($Re=100-300$)	8,640	$f=0.2891 Re^{0.9993}$, $St \approx 0.0192$	FFT script; no validation against literature; no manuscript
DeepScientist	5 ($Re=100-400$)	8,640	$f=0.4604 Re^{0.9996}$, $St \approx 0.0307$	FFT script; no validation; outline only
AI CFD Scientist	7 ($Re=60-600$)	35,156	marked <i>unresolved</i>	VLM gate; flagged case-006 anomaly; LaTeX draft preserves evidence gaps

Table 14: T3 (custom viscosity) artifact evidence under GPT-5.5. All three frameworks generated and compiled C++ libraries case-local. The differentiator is breadth and validation depth.

Framework	Cases run	Custom library compiled	Variants explored	Validation / paper artifact
ARIS	2 (1 ref + 1 custom)	libcustomViscosity.so	1 power-law ($n=0.5$)	variant comparison vs. Newtonian only; markdown summary
DeepScientist	2 (1 ref + 1 custom)	libcustomViscosity.so (variant)	1 power-law ($n=0.5$)	variant technical-report markdown with one figure
AI CFD Scientist	6 (1 ref + 5 custom)	libcustomViscosity.so	5-variant sweep ($k, n, \nabla p$)	Newtonian degeneracy ($n=1$) reproduced; nested-metadata gap preserved

Table 15: T4 (custom SA modifier) artifact evidence under GPT-5.5. All three frameworks compiled and executed a case-local custom OpenFOAM SA library implementing the requested APG production multiplier. The differences are in completeness of the surrounding pipeline: APG=0 control-case validation, DNS overlay rendering, and manuscript output.

Framework	Cases run	Custom library compiled	Reported metrics vs. DNS	Validation / paper artifact
ARIS	2 (baseline + 1 custom)	libStrainRotationSA	RMSE: 0.00430 → 0.00433	no APG=0 control; one figure; no manuscript
DeepScientist	baseline + 2 custom variants	libSAProdMult	RMSE $\approx$ 0.00433	no APG=0 control; partial report; no manuscript
AI CFD Scientist	6 (1 ctrl + 5 APG)	libCustomSA	RMSE: 0.003268 (ctrl) → 0.003258 (best, $\beta=0.15$)	APG=0 control validates code path; DNS-aligned C_f overlay; LaTeX draft

E What AI CFD Scientist Did Well, Per Task

Table 16 consolidates the per-task strengths summarized in Section 4.1. Each row is grounded in a specific archived artifact (study JSON, requirement file, run directory, VLM judgment, figure manifest, source-code library, or manuscript fragment).

F What AI CFD Scientist Does Not Yet Do Well

The strengths in Table 16 are real, but each GPT-5.5 task also exposed concrete limitations AI CFD Scientist recorded conservatively rather than papering over (Table 17). Most residual failures are in cross-experiment post-processing (parser fragility, reattachment-extraction sign convention, spectral-metadata reconstruction), not in solver execution, custom-model compilation, or the VLM gate itself.

Table 16: *AI CFD Scientist*GPT-5.5 strengths per task. Each row is supported by archived artifacts (study JSON, requirements, run directories, VLM judgments, figures, code, manuscript fragments).

ID	Task	What <i>AI CFD Scientist</i> did well (GPT-5.5)
T1	BFS sensitivity	Literature-aware ideation; mesh-gate; four-closure execution; VLM physics gate flagged the C_f post-processor and triaged a $k-\varepsilon$ output as inconsistent rather than ranking closures from suspect numbers.
T2	Jet/plume Re-sweep	Generated and validated 7 requirements; uniform mesh across the sweep; conservative <i>unresolved</i> verdict on spectral metrics rather than emitting an unsupported correlation.
T3	Custom viscosity	Generated and compiled <code>libCustomViscosity.so</code> case-local; ran 6-case study; Newtonian degeneracy reproduced ($n=1$); preserved the metadata-parser gap in the writer rather than fitting a (k, n) correlation through unlabelled points.
T4	Custom SA modifier	Generated and compiled <code>libCustomSA.so</code> ; APG=0 control matched built-in SA; rendered C_f vs. DNS overlays; reported per-case C_f RMSE with marginal differences across APG variants.
T5	Open-ended discovery	44-iteration autonomous discovery; identified outlet-region error pocket; proposed and refined Gaussian-patch source structure; 7.89% C_f RMSE reduction vs. DNS ; delivered the model as a coded <code>fvModels</code> runtime block.

Table 17: Residual limitations and *AI CFD Scientist*’s response. Each row corresponds to a verifiable artifact in the run archive.

ID	Task	Residual limitation	<i>AI CFD Scientist</i> ’s response
T1	BFS sensitivity	Reattachment x_r/h extracted with sign error.	Flagged the post-processor as suspect; declined to issue a closure ranking from the affected QoI.
T2	Jet/plume Re-sweep	Cross-experiment metadata parser could not reconstruct Re/U_b /slot width / full probe time series for several cases; case-006 centreline collapse not investigated.	Marked $f(Re)$, $St(Re)$ as <i>unresolved</i> ; preserved evidence gaps in the manuscript.
T3	Custom viscosity	Nested (k, n) metadata-parser failed for some sweep points, leaving the rheology-coefficient trend partially labelled.	Reported only the validated Newtonian degeneracy ($n=1$) and the labelled partial sweep; declined to issue a (k, n) -coefficient correlation.
T4	Custom SA modifier	Only one mesh resolution tested.	Reported control-case validation and qualitative APG sensitivity; withheld a quantitative ranking.
T5	Open-ended SA discovery	Final wall-shear / C_f extraction failed for the six post-discovery validation cases; transfer to other Reynolds numbers and geometries not tested.	Classified the result as a <i>candidate model</i> pending post-processing recovery and transfer testing; archived discovered model and full trajectory.

G Failure-Mode Taxonomy and Detection Gates

CFD automation fails along distinct axes that require different gates. Table 18 formalizes the taxonomy used by the framework. The central design point is that detection should happen at the stage where the failure becomes observable, rather than collapsing everything into a single executable/non-executable bit. The VLM physics gate exists precisely because evidential failures are invisible to the validator and to the solver log.

Table 18: Failure-mode taxonomy used by *AI CFD Scientist*. Each class is detected at a different stage and triggers a different recovery action.

Class	Typical symptom	Detector	Automatic response	Residual human task
Specification	missing solver intent, inconsistent units, incomplete BCs, plotting instructions leaking into requirements	requirement validator + deterministic cleanup	rewrite into a single executable paragraph; strip viz mentions	confirm repaired requirement still reflects scientific intent
Numerical	solver crash, divergence, unstable controls, non-physical run status	Foam-Agent logs + interpreter feedback	retry, revise requirement, or borrow stable patterns from a nearby working case (sweep-preserving)	judge whether numerical repair changed the experiment
Evidential	empty plots, wrong variable, bad framing, zoom hides phenomenon, geometry mismatch in field render	VLM physics gate (this work)	regenerate figures with revised script / framing; rerun if the gate detects geometry/topology mismatch	verify visually acceptable figures are also the right diagnostics
Narrative	unsupported claims, sparse references, missing failure cases, compilation errors in draft	reviewer prompt + pdf-latex compile loop	revise structure, references, figures, claims before accepting	expert scientific editing and sign-off

H Architectural Details: Agent Inventory and State Schema

This appendix documents the agents that implement the pathways described in [Section 3](#) and the LangGraph state object they share. [Table 19](#) lists each agent’s primary inputs, outputs, and functional role; every handoff is both human-readable and machine-readable. [Table 20](#) lists the principal fields of the checkpointed state, which are intentionally redundant: the requirement records what *should* be run, the case directory records what was *actually* run, the figures expose whether the result is physically interpretable, and the writer receives the whole artifact graph.

Table 19: Agents in *AI CFD Scientist*, their inputs, outputs, and functional role. Every handoff is both human-readable and machine-readable.

Agent / module	Primary inputs	Primary outputs	Functional role
Ideation Agent	topic, literature bundle, experiment budget	study JSON (solver, objective, experiments[], post) + novelty verdict	convert a broad topic into a concrete, bounded CFD study, avoiding overlap with retrieved prior work
Specification Agent	study JSON, selected experiment, run-topic constraints	single-paragraph <code>user_requirement</code> validation history	translate one experiment into an executable requirement; validate and repair
Mesh-Independence Gate	baseline mesh spec + refined-mesh recipe	<code>selected_mesh_spec.json</code> , percent-difference table	confirm baseline mesh is sufficient; flag for GCI escalation if needed
Foam-Agent execution	validated requirement, optional mesh assets	OpenFOAM case folder, solver logs, run status	generate dictionaries, run, low-level error correction
Code-Modification Agent	source-edit plan, equations, starter case	C++ files under <code>customModels/</code> , build system, dictionary edits, smoke run	translate physics description into a case-local OpenFOAM library
Visualization Planner / Creator	user requirement, foam case, requested figure types	PyVista/matplotlib scripts and PNG figures	produce diagnostic and paper-ready figures with traceback-driven repair
ResultsInterpreter Agent (VLM gate)	requirement, figure set, log tail	interpretation JSON: <code>simulation_success</code> , <code>requirement_met</code> , <code>issues</code> , <code>rerun_required</code> , <code>key_metrics</code>	multimodal physics verification
RerunAnalysis Agent	current requirement, interpreter feedback, nearby working-case summary	revised requirement + validator verdict	repair failing requirements while preserving the sweep dimension
OED Orchestrator	active hypothesis, artifacts so far, comparator score, budget	next action: source edit / parameter change / rerun	open-ended discovery loop
Analysis Agent	study topic, experiment bundle, per-run figures	cross-experiment visualizations + synthesis text	cross-case paper-ready figures and trend summary
Writer Agent	topic, literature, interpretations, figure bundle, analysis	LaTeX manuscript, review reports, revised PDF draft	draft the paper, compile, critique, revise
Reviewer Agent	compiled draft + compile log + reference report	pass/fail JSON + actionable recommendations	enforce formatting, claim–evidence alignment, ≥ 20 references, redundancy

Table 20: Selected fields of the *AI CFD Scientist* state object (LangGraph checkpointed state).

Field	Type	Description
<code>topic</code>	string	user-supplied research topic
<code>lit_bundle</code>	list of records	retrieved Semantic Scholar / OpenAlex / arXiv items
<code>idea</code>	study JSON	solver, target_CFL, objective, experiments[], post
<code>novelty_score</code>	float	similarity vs. retrieved literature; triggers retry if too high
<code>requirements</code>	list of strings	per-experiment validated requirement paragraphs
<code>validation_history</code>	list of records	each repair attempt with verdict and reasons
<code>mesh_spec</code>	JSON	selected mesh spec from mesh-independence gate
<code>run_results</code>	list of records	per-case <code>run_result.json</code> (status, case_dir, errors, loop_count)
<code>figs_manifest</code>	list of records	generated figures with provenance
<code>interpretations</code>	list of JSON	VLM gate output per case
<code>rerun_queue</code>	list of records	cases with <code>rerun_required=true</code> and revision plan
<code>code_mod_plan</code>	JSON	source-edit plan, files, classes, registration
<code>compile_log</code>	string	build output for case-local libraries
<code>oed_trajectory</code>	list of records	<code>iter_NNN</code> : hypothesis, action, score, status
<code>analysis</code>	JSON + figs	cross-case synthesis
<code>paper_draft</code>	LaTeX + PDF	writer + reviewer outputs across revision rounds

I LLM Cost: Token Usage and USD per Framework

We log every LLM call for every framework via the same shared accounting middleware (`llm_token_usage.json` in each run directory; `provider_usage` reporting where available). [Table 21](#) reports per-experiment token usage and estimated USD cost under matched GPT-5.5 (Codex). The reported numbers are the production end-to-end costs of running the four standard CFD experiments (BFS turbulence sensitivity, jet/plume Re-sweep, custom viscosity, custom SA modifier) on *AI CFD Scientist*, ARIS, and DeepScientist, together with the additional open-ended-discovery experiment that only *AI CFD Scientist* supports. We separate three token classes that the provider bills differently: **Input** is the uncached input the model has to read fresh; **Cached** is prompt-cached input that the provider replays at a heavily discounted rate; and **Output** is what the model actually generates. The dollar figure in the rightmost column is the user-facing bill under standard cached-input discounts.

Pricing assumptions. We compute USD using a representative codex-class price of \$1.25 per 1M uncached input tokens, \$0.125 per 1M cached-input tokens (the standard $10\times$ cached-input discount), and \$10.00 per 1M output tokens. Token counts are as recorded by the provider (`token_source: provider_usage`). The *AI CFD Scientist* runs do not exercise prompt caching, so its **Cached** column is zero by construction; ARIS and DeepScientist push large cache-replay volumes through their long-context execution loops, which is why their **Cached** columns dominate the token shape but enter the bill at the discounted rate.

Table 21: LLM cost per framework per CFD experiment under matched GPT-5.5 (Codex). *Input* is uncached input; *Cached* is prompt-cached input billed at the standard $10\times$ discount; *Output* is generated output. Cost (USD) is $1.25 \times \text{Input}/10^6 + 0.125 \times \text{Cached}/10^6 + 10.0 \times \text{Output}/10^6$. *AI CFD Scientist* does not exercise prompt caching, so its **Cached** column is zero.

Framework	Experiment	Input	Cached	Output	Calls	Cost (USD)
<i>AI CFD Scientist</i>	BFS turb. sensitivity	1,685,719	0	961,518	616	11.72
<i>AI CFD Scientist</i>	Jet/plume Re-sweep	1,010,295	0	470,049	421	5.96
<i>AI CFD Scientist</i>	Custom viscosity	1,743,752	0	968,542	595	11.87
<i>AI CFD Scientist</i>	Custom SA modifier	2,122,953	0	898,340	1,039	11.64
<i>AI CFD Scientist</i>	Open-ended discovery	1,500,481	0	69,104	94	2.57
<i>AI CFD Scientist</i>	Total	8,063,200	0	3,367,553	2,765	43.75
ARIS	BFS turb. sensitivity	6,745,470	18,060,826	68,092	131	11.37
ARIS	Jet/plume Re-sweep	6,526,605	17,845,146	66,310	128	11.05
ARIS	Custom viscosity	5,063,771	16,412,570	57,252	108	8.95
ARIS	Custom SA modifier	6,163,327	17,486,362	65,198	123	10.54
ARIS	Total	24,499,173	69,804,904	256,852	490	41.92
DeepScientist	BFS turb. sensitivity	1,314,423	46,122,554	116,694	131	8.58
DeepScientist	Jet/plume Re-sweep	1,314,423	46,122,554	116,694	129	8.58
DeepScientist	Custom viscosity	1,314,461	50,159,942	126,344	137	9.18
DeepScientist	Custom SA modifier	1,314,588	66,820,929	161,845	169	11.61
DeepScientist	Total	5,257,895	209,225,979	521,577	566	37.94

Reading the cost table. Under user-facing pricing with the standard cached-input discount applied, the three frameworks complete the same four CFD experiments at very similar dollar cost: *AI CFD Scientist* at \$41.19 (T1–T4), ARIS at \$41.92, and DeepScientist at \$37.94 — a comparable \$38–\$42 envelope. The cost comparison is therefore on a level playing field; the capability and rubric differences in [Tables 4](#) and [5](#) are not bought with extra LLM spend. What *is* different is the underlying token economy. *AI CFD Scientist* spends through many short, fully-uncached calls (**2,765** discrete LLM calls, no prompt caching, budget split roughly 2.4:1 between uncached input and generated output): every node handoff is a discrete call with an explicit JSON contract, so the same dollars buy a much higher granularity of expert-written agents. ARIS’s bill is dominated by a long-context replay-heavy execution loop that pushes $\sim 70\text{M}$ tokens through prompt caching across only 490 calls. DeepScientist’s bill is even more cache-replay-heavy: $\sim 209\text{M}$ cache-replayed tokens carrying its persistent SciMaster-style scaffolding, across 566 calls. The *AI CFD Scientist* open-ended-discovery experiment added only \$2.57 to the framework total: the OED loop hits a deterministic comparator (not the LLM) for most of its work, so OED scales with solver time, not with token cost.

Scope. These numbers cover only the production end-to-end CFD runs reported in [Sections 4 and 5](#). The VLM-ablation sweep ([Section J](#)) is excluded because each call is a single-shot vision query whose total cost is below \$1 across the 19 calls in the sweep.

J VLM Physics-Verification Gate: Planted-Failure Ablation

We quantify the value of the VLM physics-verification gate with a controlled planted-failure ablation. The retrospective on the four production GPT-5.5 runs (`scripts/inventory_decisions.py`) showed the VLM gate caught 7/21 silent failures *on top of* Foam-Agent’s own crash detection — a 33% catch rate over runs that already passed the solver-level reviewer loop. The ablation in this appendix asks the more precise question: *which kinds* of silent failure does the VLM gate catch, with what per-category recall, and at what cost?

J.1 Setup: 4 categories × 4 flows + 4 controls

We seed the ablation with four template cases that had each been PROCEED’d in production: `jet` (oscillating jet), `bfs` (backward-facing step), `hill` (periodic hill), and `chan` (channel). Each template is read-copied and one file-system-level perturbation is applied per case, drawn from a 4-bucket failure taxonomy distilled from the retrospective catches ([Table 22](#)). This gives 16 planted-failure cases plus 4 unperturbed clean controls, for 20 cases total. The verifier (`scripts/quick_interpret.py`) is the production single-shot vision-LLM call using `interpretation_system_prompt + interpretation_user_prompt` from `prompts/prompts.yaml` verbatim, returning `{PROCEED, REVISE, RERUN}`; *flagged* = REVISE ∨ RERUN.

Why post-hoc perturbation rather than a feature-disable ablation. Planting failures from already-PROCEED’d production cases makes ground truth deterministic: the (requirement, case-state) pair is unambiguously FAIL or OK because the only thing that changed from a passed case is the perturbation. This isolates the verifier’s sensitivity from confounding solver-side noise that a re-run feature-disable ablation would introduce.

Table 22: VLM-ablation design. Four flow templates (columns) × four planted-failure categories (rows) + four clean controls = 20 cases. Each cell is a single file-system perturbation applied to a read-copy of a production-passed case.

Category ↓ / Flow →	jet	BFS	hill	channel
<code>missing_deliverable</code>	delete <code>postProcessing/jetProbe</code>	delete <code>postProcessing/wallShear</code>	delete <code>postProcessing/wallShear</code>	delete latest 2 time dirs <code>\$HOME/5000s</code>
<code>wrong_magnitude_metric</code>	requirement asserts unattainable target	—	—	—
<code>broken_postprocessing</code>	zero-out 2 jetProbes files	zero-out wall-shear data	zero-out wall-shear data	zero-out U, p at $t=5000$
<code>convergence_not_settled</code>	truncate to $t \leq 0.5$ (was 8)	truncate to $t \leq 200$ (was 2000)	truncate to $t \leq 500$ (was 5000)	truncate to $t \leq 500$ (was 5000)
control (clean read-copy)	—	—	—	—

J.2 Results

[Table 23](#) reports overall confusion-matrix metrics and per-category recall. The verifier achieves **100% recall** on the three "did-the-right-thing-happen" buckets (`missing_deliverable`, `wrong_magnitude_metric`, `broken_postprocessing`) and **50% recall** on `convergence_not_settled`, for an overall recall of $14/16 = 87.5\%$ ($F1 = 82.4\%$). The two missed convergence cases (`jet_unconv`, `chan_unconv`) had `controlDict.endTime` edited to match the truncated state, so the figures look “complete to endTime” — nothing in the prompt asks whether `endTime` is *physically* sufficient for the flow to settle. Per-flow recall is uniform across geometries (jet 3/4, BFS 4/4, hill 4/4, channel 3/4); both FNs are convergence cases.

Cost. Mean wall-clock per case is 76.6 s (range 64–89 s) with one LLM call per case (19 calls total, ≈ 24 min for the full sweep) — about an order of magnitude cheaper than the production `interpret.py` loop, which regenerates figures with `viz_creator` and averages 10–15 min and 2–11 calls per case.

Table 23: VLM-ablation results: overall confusion matrix and per-category recall on the planted failures. $flagged = REVERSE \vee RERUN$. The verifier is the production single-shot call.

Ground truth				Category	N	TP	FN	Recall
	FAIL (planted)	OK (control)	total					
flagged	TP = 14	FP = 4	18	missing_deliverable	4	4	0	100%
not flagged	FN = 2	TN = 0	2	wrong_magnitude_metric	4	4	0	100%
total	16	4	20	broken_postprocessing	4	4	0	100%
Recall = 14/16 = 87.5%; Precision = 14/18 = 77.8%; F1 = 82.4%				convergence_not_settled	4	2	2	50%
				planted total	16	14	2	87.5%

Caveat on precision. All four clean controls were flagged REVERSE, giving a 77.8% overall precision. Inspection of the requirement strings shows the generic control-template requirement explicitly mentions a deliverable (e.g. a probe spectrum) that the VLM correctly notes “is not visible in the figures” — because the ablation harness only renders a small interpret-mode subset of figures, not the full reporting suite. In production, the rendered figure set is broader and the requirement is grounded in the actual case spec, so the same misalignment does not occur. *The published precision is therefore a lower bound dominated by the control template’s under-specified figure set; we report it as-is rather than back out a higher number.*

J.3 What the ablation tells us

(1) The gate catches what the solver structurally cannot see. 100% recall on `missing_deliverable` / `wrong_magnitude_metric` / `broken_postprocessing` (12/12) covers exactly the failure modes that pass Foam-Agent’s reviewer loop because the solver completed cleanly. This is the operational justification for treating the VLM gate as a first-class subsystem rather than an optional post-hoc check.

(2) Convergence-not-settled is a known blind spot. The verifier reasonably calls truncated, internally-consistent runs as PROCEED because nothing in the prompt asks whether the chosen `endTime` is physically sufficient. The actionable fix is a deterministic residual-plateau / QoI-drift detector run *before* the VLM call.

(3) Failure detection is geometry-independent. Per-flow recall (3-4 of 4 across jet, BFS, hill, channel) is statistically indistinguishable; the verifier generalizes across flow types rather than relying on memorized priors for any one canonical case.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction claim (i) an AI scientist for CFD with end-to-end discovery, (ii) source-level code modification, (iii) mesh-independence gating, (iv) a VLM physics-verification gate, and (v) an open-ended discovery pathway. [Section 3](#) describes each subsystem, [Section 4](#) reports five end-to-end CFD studies on a single GPT-5.5 backbone (including the quadrupolar SA correction reducing C_f RMSE versus DNS by 7.89% at $Re_h=5600$), and [Section 5](#) reports the head-to-head comparison against ARIS and DeepScientist. Limitations are also added to the main text [section 6](#).

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The main text includes a limitations paragraph discussing the single-backbone evaluation, manual artifact assessment, and supervised-use scope.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theorems or formal proofs. Contributions are an architecture, a domain-specific physics-verification gate, and an empirical study.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: [Section 3](#) describes the architecture and gates; [Section B](#) gives per-case quantitative tables; and [Section I](#) gives token usage and runtime per task. The code is also opensourced.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We release *AI CFD Scientist* (orchestrator, agents, prompts, run artifacts, and case directories for the five tasks) under a permissive open-source license. An anonymized release link is provided in the supplemental material at submission time. The release includes the conda environment spec, exact command lines, the custom turbulence-model libraries, and the reference datasets used for scoring. CFD baselines (OpenFOAM, Foam-Agent) are cited and version-pinned.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: All five tasks list solver, mesh, BCs, time controls, the custom code (where applicable), the comparator (e.g. C_f RMSE versus DNS), and the iteration budget for the open-ended pathway. The single LLM backbone and decoding settings are stated in [Section 4](#). No model training is performed; the LLM backbone is queried via API.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: CFD outputs are deterministic for fixed numerics, mesh, and BCs, so QoI comparisons (e.g. C_f RMSE versus DNS) do not have run-to-run variance to report. The VLM physics-verification gate is evaluated on a 20-case planted-failure set in Section 4.2, where we report the catch rate (recall) on that fixed test set rather than confidence intervals; with only 20 cases a Wilson interval would be wide and we view the headline number as a sample statistic on the constructed test set, not a population estimate.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section I reports per-task LLM token usage (prompt, cached, completion) and estimated USD cost under a caching-aware pricing schedule for GPT-5.5 (Codex). The aggregate cost envelope (\$38 to \$44 per framework on the four shared tasks) is reported in the cost appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work involves no human subjects, no scraped personal data, and no high-risk model release. The contribution is a workflow orchestrator that calls a publicly available LLM API and an open-source CFD solver. The release is anonymized at submission time.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Positive impacts: lowering the expert-time barrier for CFD studies in engineering and science, encouraging reproducible workflows by binding manuscript claims to gated artifacts, and providing a community baseline for CFD-specific scientific automation. Potential negative impacts: a more accessible CFD scientist could be misused to produce numerically stable but physically unsound studies that pass log-only checks; the paper's central VLM physics-verification gate is itself a mitigation for exactly this failure mode and the work explicitly positions human supervision as required for publication-grade use.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artifacts are an orchestrator, prompts, and CFD run directories. No pre-trained image or language model is released, no scraped dataset is included, and the framework only invokes third-party LLM APIs and OpenFOAM. No high-risk dual-use asset is introduced.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: OpenFOAM (GPLv3), Foam-Agent, the Spalart–Allmaras model, the periodic-hill DNS reference, the Driver and Seegmiller backward-facing-step experimental dataset, and the Semantic Scholar / OpenAlex / arXiv APIs are cited at first use. ARIS and DeepScientist are cited as comparison baselines.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: All assets are provided in the code repository.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work involves no crowdsourcing and no research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work involves no research with human subjects, so IRB review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: An LLM (GPT-5.5 / Codex) is the central backbone of *AI CFD Scientist*: it drives ideation, requirement specification, code generation for custom turbulence and viscosity models, interpretation of results, the VLM physics-verification gate (vision-language model variant), and manuscript drafting. The LLM is therefore an important, original component of the core method, not a writing aid. Backbone identity, decoding settings, token accounting, and cost are reported in [Section 4](#) and [Section I](#).

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.